UNIVERSITE CADI AYYAD **Année universitaire : 2026-2027**
ENSA de Marrakech
Génie Informatique

TD n° 4: Computer Vision

Filtrage des images

Dans ce TD, les fonctions OpenCV utilisables sont les suivantes :

- **cv2.blur**(src, ksize[, dst[, anchor[, borderType]]]) -> dst
- **cv2.GaussianBlur**(src, ksize, sigmaX[, dst[, sigmaY[, borderType]]]) -> dst
- **cv2.medianBlur**(src, ksize[, dst]) -> dst
- **cv2.filter2D**(src, ddepth, kernel[, dst[, anchor[, delta[, borderType]]]]) -> dst
- **cv2.getStructuringElement**(shape, ksize[, anchor]) -> retval

Afin de pouvoir tester les fonctions de filtrages, voici quelques fonctions permettant d'ajouter du "bruit" à une image selon différentes méthodes.

```
def bruitGaussien(image,moyenne,variance):
    if len(image.shape)==3:
        row,col,ch= image.shape
        sigma = variance**0.5
        gauss = np.random.normal(moyenne,sigma,(row,col,ch))
        gauss = gauss.reshape(row,col,ch)
        noisy = image + gauss
        return noisy.astype(np.uint8)
    else:
        row,col= image.shape
        sigma = variance**0.5
        gauss = np.random.normal(moyenne,sigma,(row,col))
        print(np.min(gauss),np.max(gauss))
        gauss = gauss.reshape(row,col)
        noisy = image + gauss
        return noisy.astype(np.uint8)

def bruitPoivreEtSel(image,prob):
    row= image.shape[0]
    col = image.shape[1]
    out = image.copy()
    if len(image.shape) == 2:
        black = 0
        white = 255
    else:
        colorspace = image.shape[2]
        if colorspace == 3:  # RGB
            black = np.array([0, 0, 0], dtype='uint8')
            white = np.array([255, 255, 255], dtype='uint8')
        else:  # RGBA
            black = np.array([0, 0, 0, 255], dtype='uint8')
            white = np.array([255, 255, 255, 255], dtype='uint8')
    probs = np.random.random(out.shape[:2])
    # Pepper mode
    out[probs < (prob / 2)] = black
    # Salt mode
    out[probs > 1 - (prob / 2)] = white
    return out.astype(np.uint8)

def bruitPoisson(image):
    vals = len(np.unique(image))
    vals = 2 ** np.ceil(np.log2(vals))
    noisy = np.random.poisson(image * vals) / float(vals)
    return noisy.astype(np.uint8)
```

```
def bruitSpeckle(image):
    gauss = np.random.normal(0,1,image.size)
    if len(image.shape)==3:
        row,col,ch = image.shape
        gauss = gauss.reshape(row,col,ch).astype(np.uint8)
        noisy = image + image * gauss
        return noisy.astype(np.uint8)
    else:
        row,col = image.shape
        gauss = gauss.reshape(row,col).astype(np.uint8)
        noisy = image + image * gauss
        return noisy.astype(np.uint8)
```

Exercice 1:

1. Ecrire une fonction qui prend une image et un noyau, qui utilise la convolution pour filtrer l'image en argument. Tester votre programme avec plusieurs types de filtres (vu dans le cours) ?

2. Ecrire un programme qui calcule le spectre d'une image (Transformée de Fourier) passée en argument . Visualisez le spectre dans un programme.

Exercice 2:

Ecrire un programme permettant de comparer le floutage d'une image en utilisant les 3 méthodes : **cv2.blur**, **cv2.GaussianBlur** et **cv2.medianBlur**.

Vous devrez, dans un premier temps, ajouter du bruit à une image préalablement chargée.
Votre programme devra permettre de modifier la taille du noyau de filtrage grace à un slider (la même taille de noyau sera utilisée pour les 3 filtres) ci dessous le code pour un slider:
Il est normalement impossible, dans un notebook python, de gérer les interactions utilisateur (clavier, souris, ...).
Il est cependant possible d'utiliser une bibliothèque python pour gérer des sliders

```
from ipywidgets import interact

# création de l'image
img = np.zeros((480,640,3), np.uint8)

def setColor(f):
    global img
    img[...,0] = f
    # affichage
    fig = plt.figure()
    plt.imshow(img)
    plt.show()

interact(setColor,f=(0,255,1))
```

Dans l'exemple ci-dessus, la fonction **setColor** est la fonction à appeler si la valeur du slider change.
L'appel à **interact** permet de mettre en place l'interaction. On lui passe deux arguments :

- la fonction à appeler
- la définition de la valeur d'interation. Ici f sera un slider allant de 0 à 255 par pas de 1.

Attention le nom associé lors de l'appel à **interact** doit être le même que celui de l'argument de la fonction à appeler.
Attention : notez l'utilisation du mot-clé global dans la fonction fonctionTrackbar. Sans cette ligne de code, python considérera la variable valeur comme une variable locale de la fonction et ne modifiera donc pas la valeur de la variable globale. Cette ligne n'est cependant pas nécessaire dans la fonction traitement1 puisque

valeur est utilisée en partie droite d'une affectation. Il en va de même pour la variable image. Comme elle ne fait pas l'objet d'une affectation directe (du type image=...) alors python comprend qu'il s'agit d'une variable globale.

Pour comparer les résultats, utiliser la fonction **mse** qui calcule la différence entre 2 images (doivent être aux mêmes dimensions) suivante :

```
def mse(img1, img2):
   h, w = img1.shape
   # on force le passage en double pour avoir les différences négatives
   diff = cv2.subtract(img1, img2,dtype=np.double)
   err = np.sum(diff**2)
   mse = err/(float(h*w))
   diff = np.clip(np.abs(diff),0.0,255.0)
   return mse, diff.astype(np.uint8)
```

Cette fonction prend en argument les deux images à comparer. Elle renvoie deux résultats : l'erreur moyenne et l'image des différences.

Exercice 3:

Les 3 fonctions utilisées précédemment permettent d'utiliser des filtres usuels de taille carrée (ou rectangulaire).
Il est possible d'appliquer des filtres de taille et de forme non usuelles. Pour cela, on utilise la fonction cv2.getStructuringElement. Cette fonction va renvoyer une matrice 2d dont les éléments inclus dans la forme définie par le paramètre *shape* (par exemple : ***cv2.MORPH_CROSS)***
sont initialisés à 1 alors que les autres seront à 0.
Le noyau de filtrage ainsi obtenu peut être appliqué à une image en utilisant la fonction cv2.filter2D.
Utiliser ces fonction pour appliquer un filtre moyen en croix de taille 5x5 sur une image.
Comparer les résultats avec ceux obtenus par un filtrage moyen de même taille en utilisant la fonction cv2.blur.

Exercice 4:

Cet exercice peut être traité en utilisant uniquement la librairie numpy pour le calcul de la fft, le shift etc., mais nous utiliserons opencv et numpy quand c'est possible.

Le but de l'exercice est de filtrer l'image « car.png » et l'image « lena-sinus.jpg » dans le domaine fréquentiel en utilisant un filtre passe-bas. Pour cela, on doit définir et compléter les fonctions et suivre les instruction dans le code ci-dessous :

```
// définir une fonction qui permute les quadrants, a été déjà définie TD2
fftshift(input_img, output_img)
{
}
// définir une fonction qui calcule la transformée de Fourier question 2 exo 1
calculateDFT(scr, dst)
{
}
// définir une fonction qui définie le filtre passe-bas H de diamètre D0
construct_H(scr, D0)
{
}

// définir une fonction qui applique H à la transformée de Fourier
filtering(scr, dst, H)
{
```

}

Pensez à transformer vos images en float 32 bits avant tout calcul. Pour visualisez la magnitude penser à visualiser son log *20